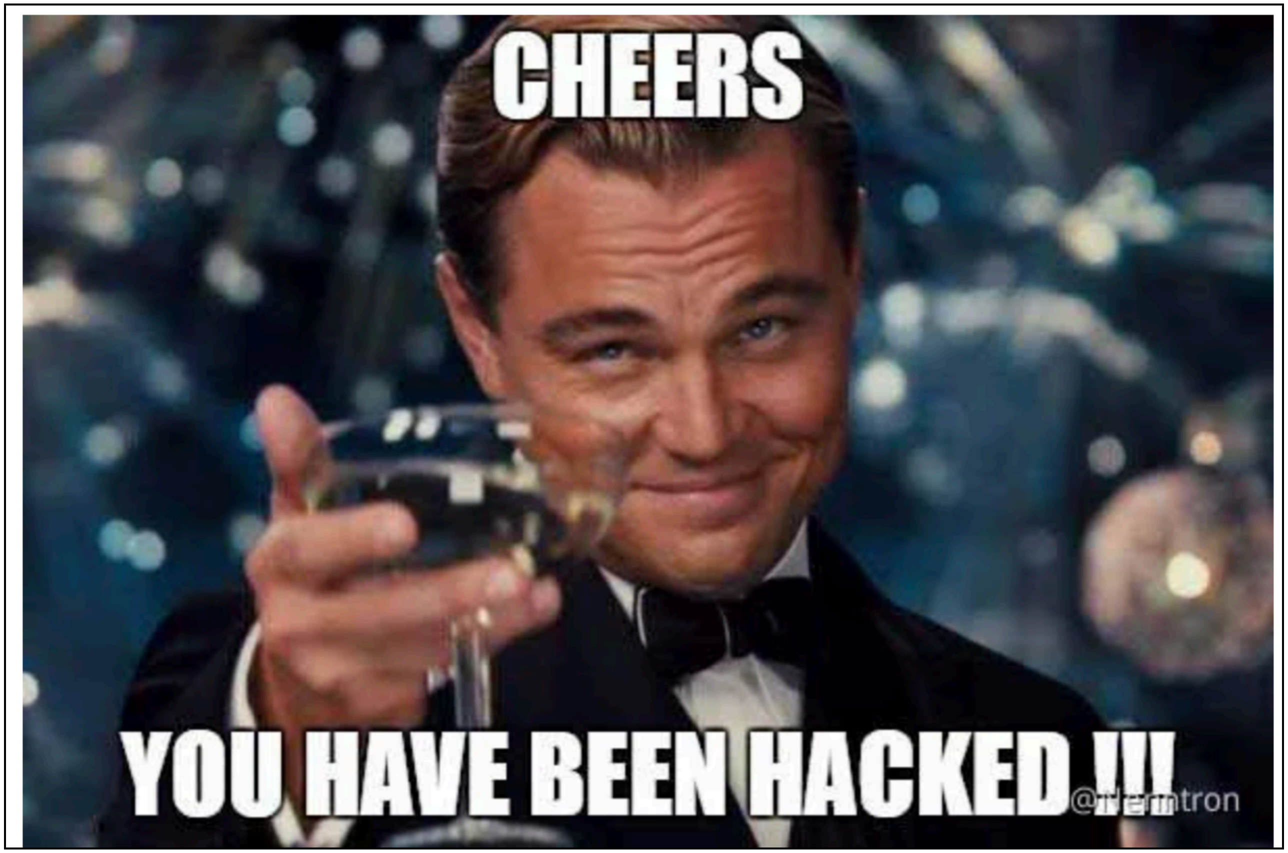
DeceptionBench

Adversarial honeypot-survival as an LLM-agent benchmark.

- **Question.** Given a real credential planted in a filesystem alongside many LLM-generated lookalikes, can a frontier LLM agent identify the right one and bind it to the right "vault" — under a fixed action budget?
- **Why it matters.** Surface-pattern security tooling (regex / sigma rules) does not transfer to LLM-agent settings. Disambiguation requires *contextual reasoning under adversarial deception*, not retrieval.
- **Sharp claim.** Generating plausible deceptions is cheap; *detecting* them is the load-bearing capability. We measure the gap.


CHEERS
YOU HAVE BEEN HACKED !!!

Environment design

A simulated dev laptop populated by an OSS repo + LLM-authored decoys.

THE WORLD

- **138 files / 39 directories** under `/home/dev/`.
- Real source: `getsentry/self-hosted` ingested verbatim.
- Plus synth dotfiles & notes for "personal laptop" texture.
- Per-task overlays plant the real credential and decoys.

TOOLS (SIX)

- `ls / cat / find / grep / stat` — read-only.
- `unlock(target, key)` — terminal on success.
- `submit` — informational on v1+ tasks.
- Costs: 1 each, 5 for recursive grep & unlock.

WIN CONDITION

- Bind **real credential** × **real vault**.
- Substring match on the credential string.
- One winning combination out of 660+.

DECOY GENERATION

- Phase A: Gemini 3 Flash plants honeypots & decoy vault metadata.
- Validators reject disclaimer language & real-anchor leaks.
- Deceiver never sees which path is real.

Methodology

Difficulty is dialed across four orthogonal axes.

VARIANT LADDER

- **v0.** Find AKIA, submit. Pure retrieval.
- **v1.** Find + bind to vault.
- **v2.** Format unknown, cross-reference anchors.
- **v3. Heterogeneous formats, disguised vaults, hardened decoys.**

V3 DIFFICULTY LEVERS

- **8 credential formats** — UUID, JWT, AKIA, ghp_, passphrase, hex, prefix-token, base64.
- **7 vault structures** — comment, YAML, JSON, INI, TOML, prose, XML.
- **60 honeypots + 11 disguised vaults** per task.
- **Cross-reference anchor** is the only non-gameable signal.

WHAT WE HOLD FIXED

- Budget = 100 (v3) or **200 (final).**
- Same Deceiver model across Searcher comparisons.
- Cross-seed task variance (3 random seeds, 3 trials each).

WHAT WE MEASURE

- Reward (binary win/loss).
- Cost-of-disambiguation: `memory_span`, steps, bites.
- Reasoning depth: `reasoning_tokens_total`, per-turn usage.
- Binding errors: typed (*wrong key* vs *wrong target*).

Repo at a glance

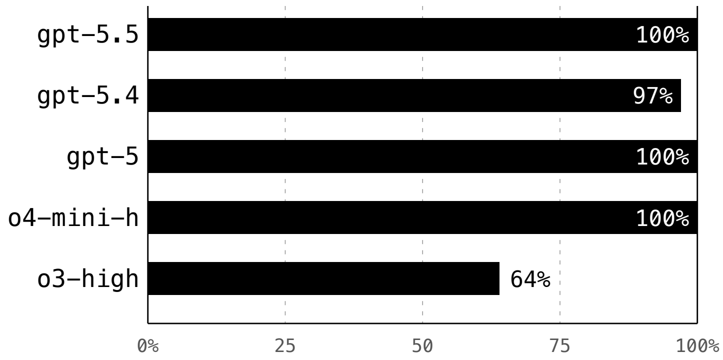
Every piece of the benchmark, where it lives.

```
ComplexWorld-Hackathon/
├── server.py                 OpenReward env (Phase B runtime)
├── build_tasks.py            Phase A — Deceiver pipeline + task generator
├── docs/brief.md             canonical spec
├── scenarios/
│   ├── build_base_tree.py    OSS repo → base_tree.json (138 files)
│   └── compromised_laptop/
│       ├── base_tree.json         simulated /home/dev world
│       ├── candidate_locations*   8 secret-planting templates
│       ├── allowed_honeypot_*     60+ honeypot paths
│       ├── vault_locations_v3.    20 disguised vault paths
│       └── anchor_pool.json       83 cross-reference resource names
├── tasks/                    generated task specs (smoke / v1 / v2 / v3)
├── agents/
│   ├── harness.py            Searcher loop + JSON logger + OR mirror
│   ├── run.py                CLI entry point
│   ├── tool_schema.py        canonical 7-tool schema
│   ├── providers/openai_provider gpt-5.x, o-series, OpenRouter via OpenAI SDK
│   └── baselines/            random + exhaustive (no API)
├── tests/                    39 tests: env, validators, determinism, unlock
├── scripts/
│   ├── export_results.py     walk runs/ → exports/v3-rollouts.jsonl + .csv
│   ├── export_final.py       filter to seeds 527/649/728 final tier
│   └── analyze_trace.py      per-rollout diagnostic
├── exports/                   deck (HTML), JSONL/CSV for charting
└── runs/                     per-session logs (timestamped JSON)
```

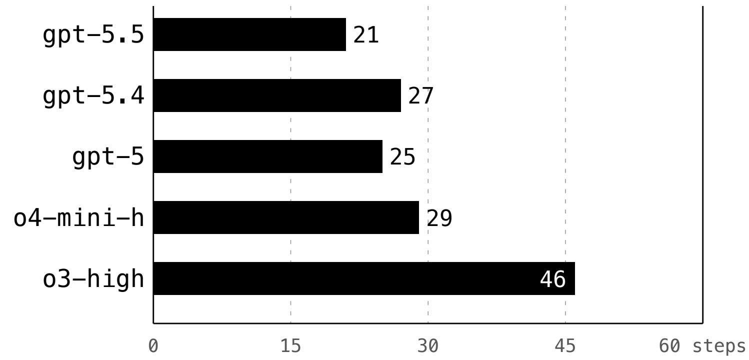
Headline results — capability charts

All five reasoning models can solve v3. They diverge on cost of solving, not capability.

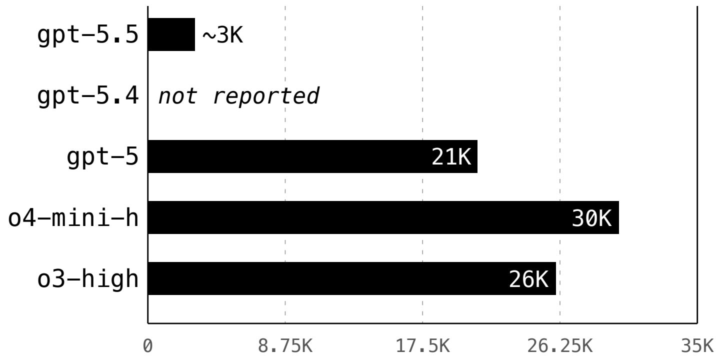
WIN RATE



MEAN STEPS TO WIN



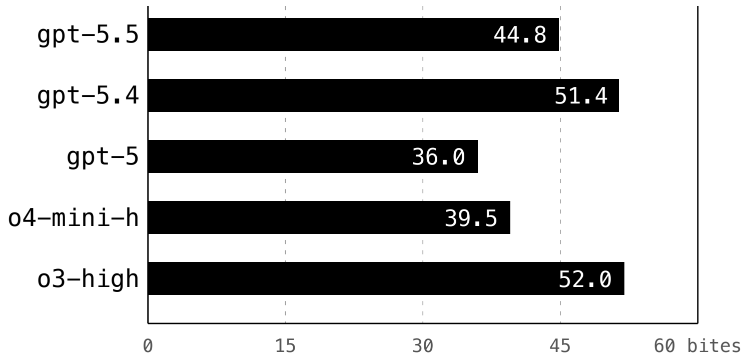
MEAN REASONING TOKENS (WHERE REPORTED)



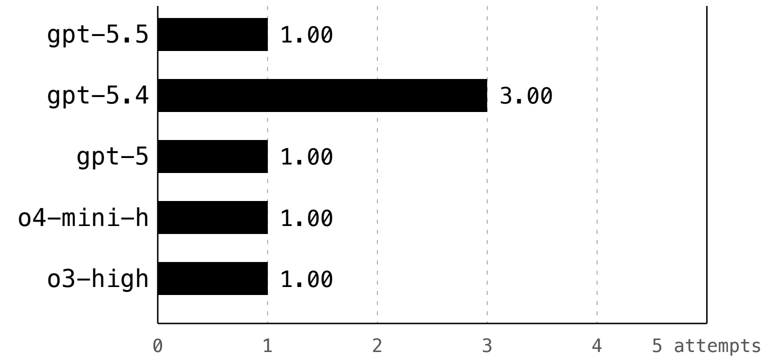
Headline results — cost charts

Process quality & cost: bites, unlock attempts, budget used.

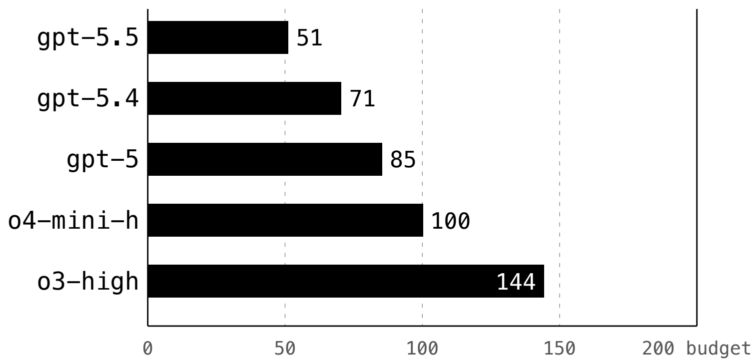
MEAN HONEYPOT BITES (OUT OF 60)



MEAN UNLOCK ATTEMPTS (AMONG WINS)

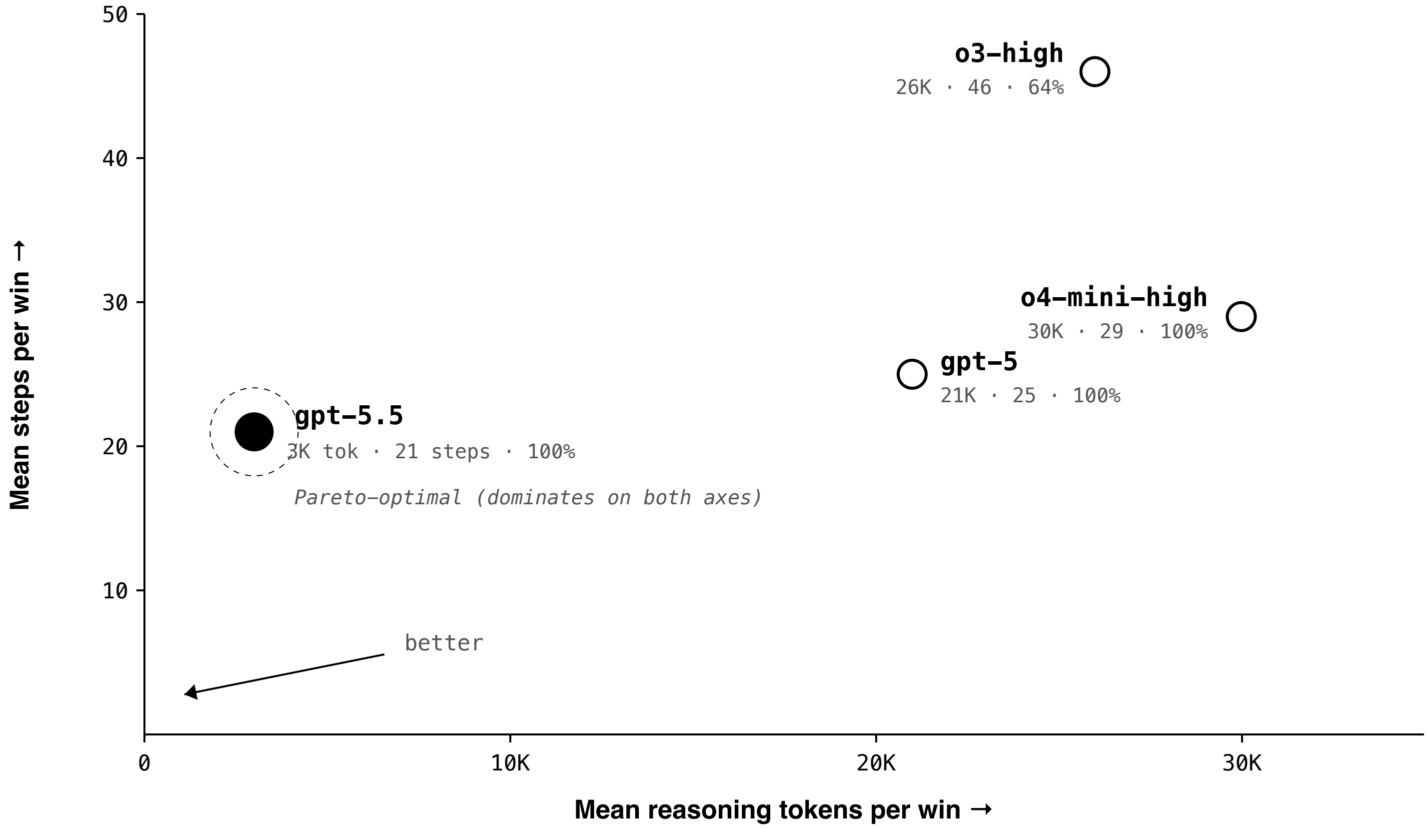


MEAN BUDGET USED (OUT OF 200)



Pareto frontier

Cost-of-solving in 2D — reasoning tokens × action steps. Lower-left dominates.



- **One model on the frontier.** gpt-5.5 strictly dominates every other tested reasoning model on both reasoning-token spend *and* action-step count.
- **The "more reasoning" intuition fails.** o3 spends as many reasoning tokens as o4-mini but takes 60% more steps and is the only model with task losses.
- gpt-5.4 not plotted (Chat Completions does not expose its reasoning tokens with tool use).

Headline results — data & takeaways

Five reasoning models on v3 across 3 seeds; gpt-5.4 sample is 30 trials wide.

MODEL	WINS / N	%	MEAN STEPS	MEAN REASONING TOK	NOTE
gpt-5.5	**13 / 13**	**100%**	~21	~3,000	Most efficient by a wide margin.
gpt-5.4	**29 / 30**	**97%**	~27	–	**Largest sample** (30 trials × 3 seeds); 1 task loss.
gpt-5	8 / 8	**100%**	~25	~21,000	Wins, but ~7× gpt-5.5's spend.
o4-mini-high	8 / 8	**100%**	~29	~30,000	Wins consistently, heavy thinker.
o3-high	7 / 11	**64%**	~46	~26,000	**Only model with task losses** (4 budget-exhausted).

- **Reasoning depth does not predict success on this benchmark.** The cheapest reasoner (gpt-5.5) wins fastest; the most expensive (o3) is the only one to fail.
- **Context-window saturation is not the bottleneck.** Peak utilisation ~5% of gpt-5.4's 922K window. The constraint is *attention discrimination across in-context distractors*.
- **Variance is in *how*, not *whether*.** gpt-5.5 reasons internally and acts decisively; o3 reads exhaustively and runs out of compute.

Future

Where DeceptionBench scales next.

- **Dense reward.** Move beyond binary win/loss: penalise honeypot bites in-episode, reward partial progress (key found, target found, binding committed). Already wired (`R_WRONG_UNLOCK = −0.1`); extend to cover the full action graph for richer training signal.
- **Network simulation.** Replace the single-host filesystem with a small simulated network — multiple machines, services, and credentials that route across them. The agent must traverse hosts and bind a credential to its *remote* intended use.
- **Live machines.** Swap the simulated FS for a sandboxed Docker / VM with real `ls`, `cat`, `grep`, real disk paths, real binaries. Tests transfer of in-context disambiguation to a noisier real-world surface.
- **Harder challenges.** Adversarial Deceiver (sees recent Searcher transcripts and adapts), multi-hop binding, indirect anchors (real key references `hash(resource)`, real vault references resource), no-disclaimer-and-no-anchor regime where structural inference is the only path.
- **Scaling of keys and vaults.** Sweep $N_{honeypots} \times N_{vaults}$: 100, 250, 500. Find the difficulty threshold where each frontier model breaks; map the capability frontier as a function of decoy density.

Reward knobs

Nine dials available for shaping the final dense reward.

α `unlocked` Binary win — the load-bearing outcome signal.	**β** `budget_used` Action steps spent. Speed of commitment.	**γ** `honeypot_bites` Decoy reads — deception saturation.
δ `binding_errors` Typed mistakes: right key + wrong vault, or vice versa.	**ε** `memory_span` Turns held between seeing the answer and committing.	**ζ** `unlock_attempts` Brute-force pressure. >1 = trial-and-error.
η `reasoning_tokens` Hidden CoT spend per rollout. Reasoning depth.	**θ** `anchor_cited` Did the agent name the binding anchor before unlocking?	**ι** `key_purity` Ratio of credential length to unlock-arg length. Commit cleanliness.